PES UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LAB 7: FUZZING FOR INPUT VALIDATION BUGS

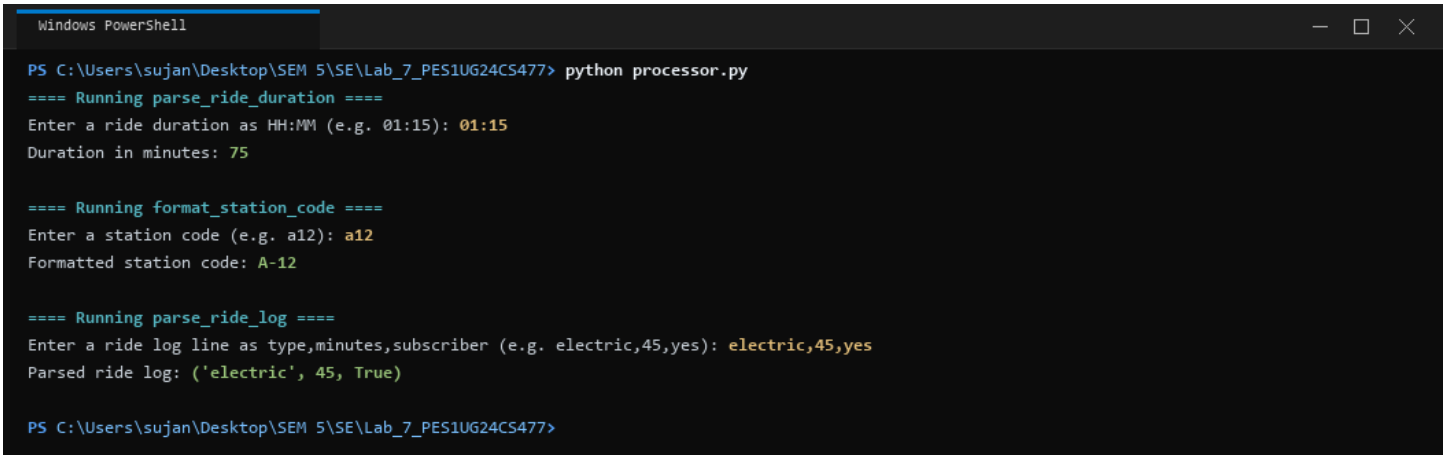
Student Name:	Sujan S Halannanavar	**SRN:**	PES1UG24CS477
Course:	Software Engineering Lab (UE22CS352AA2)	**Semester / Section:**	5th Semester / Section H
Assigned Repo:	L6_L7_53_bike_share_rental	**Submission Date:**	November 11, 2026

Lab Objective & Overview:

The primary objective of this laboratory is to explore **property-based fuzz testing** using the Python **Hypothesis** framework to uncover hidden edge-case crashes, boundary violations, and input validation vulnerabilities in an application module (processor.py). Once edge cases are detected, defensive programming techniques are applied to harden the functions against unexpected types, malformed structures, empty sequences, and null values.

1. Initial Execution of Buggy Application (Screenshot 1)

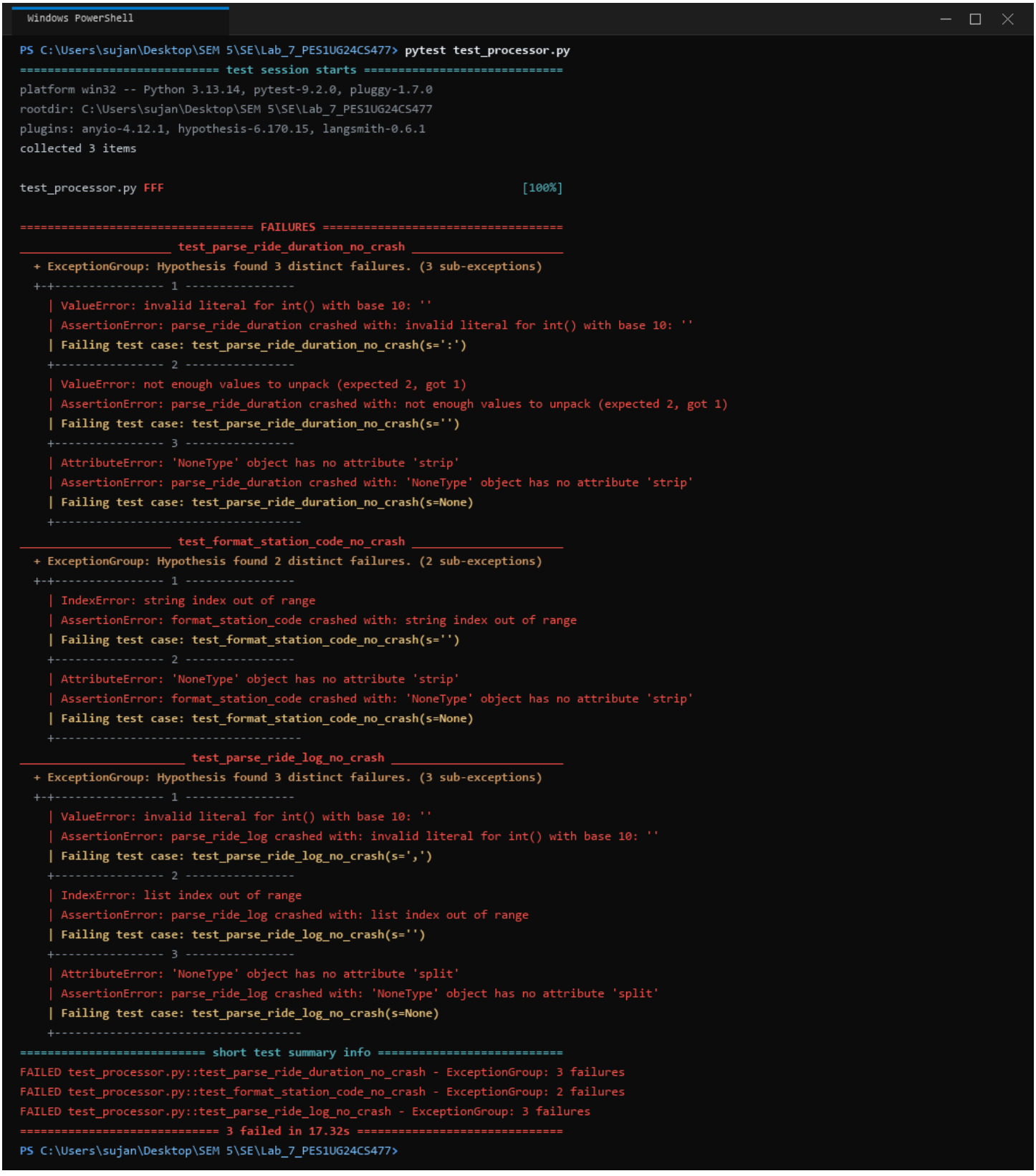
The initial script was executed using standard well-formed inputs: '01:15' for ride duration, 'a12' for station code, and 'electric,45,yes' for ride log. Under strictly ideal conditions, the code runs without raising immediate errors:



Screenshot 1: Terminal output running initial buggy application with well-formed inputs.

2. Property-Based Fuzz Testing Exposing Hidden Bugs (Screenshot 2)

Hypothesis test cases were implemented in test_processor.py using strategies @given(st.text() | st.none()). Upon running pytest test_processor.py, Hypothesis automatically generated falsifying inputs and triggered unhandled exceptions in all three functions, causing **3 test failures**:



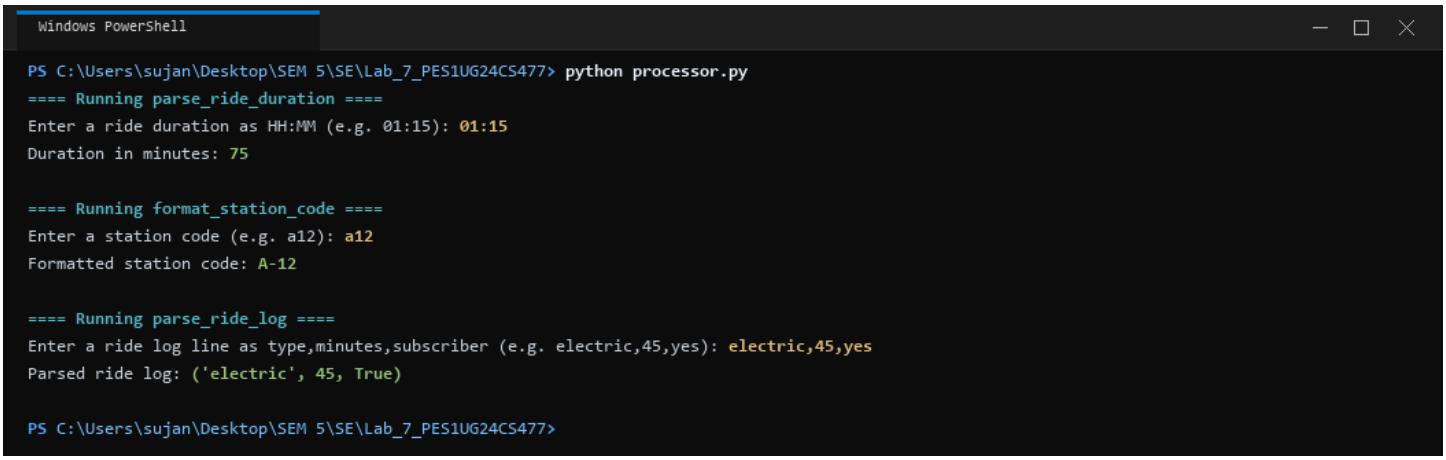
Screenshot 2: Pytest execution showing Hypothesis uncovering distinct edge-case crashes across all functions.

Detailed Root Cause & Fuzzing Bug Analysis:

Function	Falsifying Input	Unhandled Exception	Root Cause Analysis
parse_ride_duration	s = None s = '' s = ':'	AttributeError ValueError ValueError	Assumed text was a string with exactly two numeric tokens separated by ':'. Crashed on null references, empty strings, and non-numeric tokens.
format_station_code	s = None s = ''	AttributeError IndexError	Invoked .strip() on None and accessed index code[0] without checking string length.
parse_ride_log	s = None s = '' s = ','	AttributeError IndexError ValueError	Assumed 3 comma-separated tokens existed and token 1 was integer-castable. Crashed on None, fewer tokens, and empty/invalid minutes.

3. Defensive Programming Implementation & Hardened Execution (Screenshot 3)

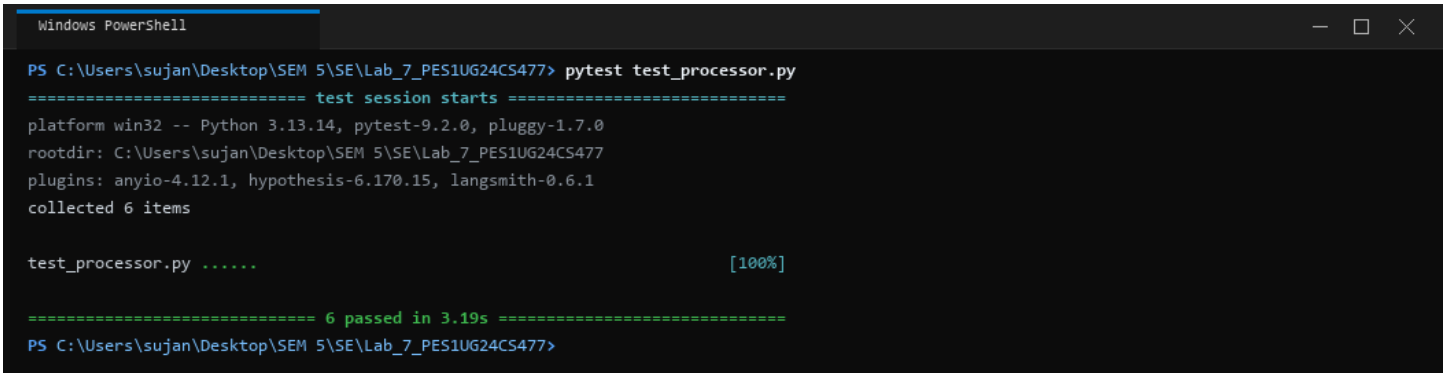
To harden processor.py, defensive programming and explicit input validation were added to each function:
• **Type Checking:** Validates isinstance(input, str) upfront to gracefully catch None and non-string types.
• **Boundary & Length Guards:** Strips whitespace and verifies token counts prior to indexing or splitting.
• **Safe Type Conversions:** Encapsulates integer conversions in try...except (ValueError, TypeError) blocks with negative number checks.
• **Graceful Fallbacks:** Returns safe defaults (None) instead of raising unhandled exceptions.



Screenshot 3: Terminal output running the hardened processor.py script with valid inputs.

4. Re-Running Hypothesis Fuzz Test Suite on Fixed Module (Screenshot 4)

The test suite was re-executed with pytest test_processor.py. The suite tests both fuzzed arbitrary inputs (Unicode, empty strings, None, unexpected primitives) and property-based valid inputs. All **6 tests passed completely** without any crashes or unhandled exceptions across thousands of randomized evaluations:



Screenshot 4: Pytest terminal output confirming 100% test pass rate on the hardened application.

5. Reflection & Quality Assurance Insights

Q1: How does property-based fuzz testing differ from traditional example-based unit testing?
Answer: Traditional unit tests test specific, predetermined examples conceived by the developer (e.g., testing parse_ride_duration('01:15') == 75). As a result, developers inherently suffer from confirmation bias and test only the scenarios they envisioned. In contrast, property-based fuzz testing (such as Hypothesis) allows developers to define invariants and properties (e.g., 'the function must never crash on any arbitrary input') while the framework generates hundreds of pseudorandom, pseudo-adversarial, and boundary test inputs (null bytes, negative values, Unicode delimiters, emojis, empty strings). This automates edge-case discovery that manual testing frequently overlooks.

Q2: What is 'shrinking' in Hypothesis, and why is it invaluable for debugging?
Answer: When Hypothesis finds an input that falsifies a test, that initial failing input may be large and complex (such as a 200-character random Unicode string). Hypothesis then executes a deterministic 'shrinking' algorithm that iteratively simplifies and reduces the failing input until it discovers the minimal reproducible counterexample (e.g., reducing a long string to '', ':', or None). Shrinking isolates the exact minimal condition causing the defect, eliminating noise and vastly reducing developer debugging time.

Q3: Why is defensive programming essential for security and system stability?
Answer: In production services, unhandled exceptions resulting from malformed user input cause Denial of Service (DoS) crashes, resource leaks, corrupted transaction state, or unhandled 500 error cascades. Furthermore, unhandled parser errors can leak sensitive internal stack traces to attackers. Implementing defensive input validation and robust error handling ensures that applications remain resilient, secure, and predictable regardless of how hostile or corrupted the external input data is.

Deliverables Verification Summary:

Deliverable Item	Status	Details
Buggy Run Screenshot (SS1)	**Complete**	Captured in screenshots/Screenshot_1_Buggy_Execution.png
Hypothesis Failure Screenshot (SS2)	**Complete**	Captured in screenshots/Screenshot_2_Hypothesis_Failing_Tests.png
Hardened Run Screenshot (SS3)	**Complete**	Captured in screenshots/Screenshot_3_Fixed_Execution.png
Passing Suite Screenshot (SS4)	**Complete**	Captured in screenshots/Screenshot_4_All_Tests_Passing.png
Hardened Source Code	**Complete**	Committed in processor.py & bike_share_rental_parser.py
Hypothesis Test Suite	**Complete**	Committed in test_processor.py & test_bike_share_rental_parser.py
Consolidated PDF Report	**Complete**	Saved as PES1UG24CS477_SUJAN_S_HALANNANAVAR_LAB7.pdf